Citibank Banking Database

SQL Analysis Case Study

Prepared by: Jyotika Ojha

Platform: MySQL / BigQuery — Dataset: citibank

Submission Deliverable — SQL Queries, Output Samples, and Insights

Overview

This document presents the SQL analysis performed on the Citibank banking database covering customer accounts, transactions, loans, cards, branches, employees, and customer service interactions. All 10 CSV files were loaded as tables into a "citibank" dataset/schema. Each question below includes the SQL query (MySQL 8 / BigQuery Standard SQL syntax), a sample of the first 10 rows of output, and a short insight describing what the result means for the business. Section VII presents consolidated, actionable recommendations synthesized across all analysis areas.

Note on the underlying data: this is a synthetic dataset generated for the case study exercise. A few structural quirks were observed during analysis (e.g., card credit_limit is populated only for credit cards, and the account "balance" field does not track a card's outstanding statement balance separately). These are called out inline wherever they affect interpretation of a result.

Data Preparation Note: Date Formats

The source CSVs store dates as DD-MM-YYYY (and DD-MM-YYYY HH:MM:SS for timestamps). When uploaded directly to BigQuery/MySQL, columns in this format are typically inferred as STRING rather than DATE/DATETIME. Before running the queries in this document, each date/timestamp column was converted once using STR_TO_DATE, for example:

```
UPDATE citibank.accounts
SET opened_date = STR_TO_DATE(opened_date, '%d-%m-%Y');

UPDATE citibank.transactions
SET transaction_date = STR_TO_DATE(transaction_date, '%d-%m-%Y %H:%i:%s');
```

All queries in this document assume opened_date, created_at, transaction_date, start_date, end_date, payment_date, issue_date, expiry_date, hire_date, and date_of_birth have already been converted to proper DATE/DATETIME columns via this pattern. (In BigQuery, PARSE_DATE('%d-%m-%Y', col) / PARSE_DATETIME('%d-%m-%Y %H:%M:%S', col) achieve the same result.)

I. Initial Exploratory Analysis

I.A — Data types for columns in the 'customers' table

SQL Query

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'citibank'
  AND table_name = 'customers'
ORDER BY ordinal_position;
```

Output (first 9 of 9 row(s))

column_name	data_type
customer_id	INT (Primary Key)
first_name	VARCHAR(50)
last_name	VARCHAR(50)
email	VARCHAR(100)
phone	VARCHAR(20)
address	VARCHAR(255)
date_of_birth	DATE
ssn	VARCHAR(11)
created_at	DATETIME

Insight: The customers table uses a standard normalized structure: an integer primary key (customer_id), text fields for identity and contact details, a DATE field for date_of_birth, and a DATETIME/TIMESTAMP field for created_at. Keeping identity fields (SSN, email, phone) as VARCHAR rather than numeric types preserves formatting (leading zeros, punctuation) and avoids unintended arithmetic.

I.B — Date range of customer accounts and transactions

SQL Query

```
SELECT 'Customer Accounts' AS record_type,
       MIN(created_at) AS earliest_date,
       MAX(created_at) AS latest_date
```

```
FROM citibank.customers
UNION ALL
SELECT 'Transactions',
       MIN(t.transaction_date),
       MAX(t.transaction_date)
FROM citibank.transactions t
JOIN citibank.accounts a ON a.account_id = t.account_id;
```

Output (first 2 of 2 row(s))

record_type	earliest_date	latest_date
Customer Accounts	2015-03-09 00:59:30	2025-12-13 16:04:35
Transactions	2020-01-02 06:04:21	2025-12-29 10:16:09

Insight: Customer accounts were created between March 2015 and December 2025, while recorded transactions span January 2020 to December 2025. Because transaction history only goes back to 2020, any customer who joined between 2015-2019 has roughly the first 1-5 years of their relationship with the bank unrepresented in the transaction table — this should be flagged to the data team before using early-tenure customers in trend analyses.

I.C — Accounts by branch, account type, and status

SQL Query

```
SELECT
  b.branch_name,
  a.account_type,
  a.status,
  COUNT(a.account_id) AS total_accounts
FROM citibank.accounts a
JOIN citibank.branches b ON b.branch_id = a.branch_id
GROUP BY b.branch_name, a.account_type, a.status
ORDER BY b.branch_name ASC, total_accounts DESC;
```

Output (first 10 of 82 row(s))

branch_name	account_type	status	total_accounts
Aventura Plaza	FIXED_DEPOSIT	FROZEN	4
Aventura Plaza	SAVINGS	ACTIVE	3
Aventura Plaza	SAVINGS	FROZEN	3
Aventura Plaza	CHECKING	ACTIVE	2
Aventura Plaza	CHECKING	CLOSED	2
Aventura Plaza	SAVINGS	CLOSED	2
Aventura Plaza	FIXED_DEPOSIT	ACTIVE	1
Aventura Plaza	FIXED_DEPOSIT	CLOSED	1
Brickell Avenue	CHECKING	CLOSED	4
Brickell Avenue	SAVINGS	ACTIVE	4

Insight: Account mix (CHECKING, SAVINGS, FIXED_DEPOSIT) and status (ACTIVE, FROZEN, CLOSED) varies widely by branch, with some branches carrying disproportionately high FROZEN or CLOSED shares. Branches with a high concentration of non-active accounts are candidates for a re-engagement or account-cleanup campaign to convert dormant relationships back into active, revenue-generating ones.

II. Customer Behavior Analysis

II.A — Do high-balance accounts transact more frequently?

SQL Query

```
WITH acct_txn AS (
  SELECT
    a.account_id, a.customer_id, a.balance,
    CASE
      WHEN a.balance > 50000 THEN 'High Balance (>50K)'
      WHEN a.balance BETWEEN 10000 AND 50000 THEN 'Medium Balance (10K-50K)'
      ELSE 'Low Balance (<=10K)'
    END AS balance_category,
    COUNT(t.transaction_id) AS txn_count
  FROM citibank.accounts a
  LEFT JOIN citibank.transactions t ON t.account_id = a.account_id
```

```
  GROUP BY a.account_id, a.customer_id, a.balance
)
SELECT
  balance_category,
  COUNT(DISTINCT customer_id) AS num_customers,
  ROUND(AVG(txn_count), 2) AS avg_transaction_count,
  ROUND(AVG(balance), 2) AS avg_balance
FROM acct_txn
GROUP BY balance_category
ORDER BY avg_transaction_count DESC;
```

Output (first 3 of 3 row(s))

balance_category	num_customers	avg_transaction_count	avg_balance
Low Balance (<=10K)	18	1.16	4882.46
High Balance (>50K)	83	1.0	77564.67
Medium Balance (10K-50K)	62	0.96	32443.73

Insight: Counter-intuitively, Low Balance accounts show the highest average transaction count (1.16), while High Balance accounts show the lowest (1.00) despite holding an average balance of ~$77.6K. This suggests high-balance customers are using their accounts as a store of value (savings/parking funds) rather than for active daily banking — an opportunity to cross-sell transaction-linked products (rewards cards, cash-management tools) to activate this segment.

II.B — Peak transaction months per account type in 2025

SQL Query

```
WITH monthly AS (
  SELECT
    a.account_type,
    MONTH(t.transaction_date) AS month_num,
    MONTHNAME(t.transaction_date) AS month_name,
    COUNT(t.transaction_id) AS transaction_count,
    SUM(t.amount) AS total_transaction_amount
  FROM citibank.transactions t
  JOIN citibank.accounts a ON a.account_id = t.account_id
  WHERE YEAR(t.transaction_date) = 2025
  GROUP BY a.account_type, month_num, month_name
),
ranked AS (
  SELECT *, RANK() OVER (PARTITION BY account_type ORDER BY transaction_count DESC) AS rnk
  FROM monthly
)
SELECT account_type, month_name, transaction_count, total_transaction_amount, rnk AS rank_num
FROM ranked
WHERE rnk <= 2
ORDER BY account_type, rnk;
```

Output (first 10 of 12 row(s))

account_type	month_name	transaction_count	total_transaction_amount	rank
CHECKING	March	3	13687.25	1
CHECKING	January	1	9662.33	2
CHECKING	February	1	9391.14	2
CHECKING	April	1	8973.11	2
CHECKING	July	1	7551.88	2
CHECKING	December	1	6697.89	2
FIXED_DEPOSIT	April	3	18265.26	1
FIXED_DEPOSIT	August	2	4581.79	2
FIXED_DEPOSIT	September	2	9514.91	2
FIXED_DEPOSIT	December	2	5852.73	2

Insight: Each account type has a distinct seasonal peak (e.g., CHECKING peaks in March, FIXED_DEPOSIT in April), suggesting different customer behaviors tied to tax season, fiscal-year timing, or promotional windows. Staffing, fraud-monitoring capacity, and marketing pushes should be aligned to these account-type-specific peak months rather than using a single bank-wide seasonal calendar.

II.C — Credit card transactions by age group (as of June 1, 2025)

SQL Query

```
SELECT
  CASE
    WHEN DATEDIFF('2025-06-01', c.date_of_birth) / 365.25 < 30 THEN 'Under 30'
    WHEN DATEDIFF('2025-06-01', c.date_of_birth) / 365.25 BETWEEN 30 AND 50 THEN '30-50'
    ELSE 'Over 50'
  END AS age_group,
  COUNT(t.transaction_id) AS credit_card_transaction_count
FROM citibank.transactions t
JOIN citibank.accounts a ON a.account_id = t.account_id
JOIN citibank.cards cd ON cd.account_id = a.account_id AND cd.card_type = 'CREDIT'
JOIN citibank.customers c ON c.customer_id = a.customer_id
WHERE c.date_of_birth IS NOT NULL
GROUP BY age_group
ORDER BY credit_card_transaction_count DESC;
```

Output (first 3 of 3 row(s))

age_group	credit_card_transaction_count
Over 50	50
30-50	37
Under 30	18

Insight: Customers Over 50 generate the most credit card transactions (50), followed by the 30-50 group (37) and Under 30 (18). Younger customers are the lightest credit card users — this points to an opportunity for starter/student credit-card products with engagement incentives to build habits and lifetime value earlier in the customer relationship.

II.D — Top 10 customers by total active-account balance

SQL Query

```
SELECT
  c.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  c.email,
  SUM(a.balance) AS total_balance,
  COUNT(a.account_id) AS num_accounts,
  MIN(a.opened_date) AS earliest_account_opened
FROM citibank.accounts a
JOIN citibank.customers c ON c.customer_id = a.customer_id
WHERE a.status = 'ACTIVE'
GROUP BY c.customer_id, c.first_name, c.last_name, c.email
ORDER BY total_balance DESC
LIMIT 10;
```

Output (first 10 of 10 row(s))

customer_id	customer_name	email	total_balance	num_accounts	earliest_account_opened
41	David Patel	smithjulieexample.org	199608.73	3	2021-06-10
154	Diana Hebert	brandon39example.net	154251.97	2	2021-12-20
94	Beverly Wagner	phyllismeyerexample.net	140071.94	2	2020-03-28
151	Joan Johnson	paulproctorexample.org	140069.64	2	2023-08-24
90	Justin Marquez	smithjasonexample.org	121257.86	2	2022-06-08
105	Loretta Barnes	lindsey69example.org	117072.95	2	2021-02-22
12	Rebecca Gardner	rodriguezsierraexample.net	98393.6	1	2023-12-21
200	Katelyn Davis	thomaskevinexample.net	96472.08	1	nan
30	Charles Harrington	jsmithexample.com	96289.33	1	nan
11	Eric Campbell	mcmillandennisexample.org	94961.17	1	nan

Insight: The top 10 customers by balance hold combined balances well above the portfolio average, with the #1 customer alone holding just under $200K across 3 accounts. This is Citibank's high-net-worth segment and should be prioritized for a dedicated relationship-manager / private-banking track, since losing even one of these customers has an outsized revenue impact.

II.E — Customers with more than one active loan

SQL Query

```
SELECT
  c.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  COUNT(l.loan_id) AS num_active_loans,
  SUM(l.loan_amount) AS total_loan_amount
FROM citibank.loans l
JOIN citibank.customers c ON c.customer_id = l.customer_id
WHERE l.status = 'ACTIVE'
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(l.loan_id) > 1
ORDER BY total_loan_amount DESC;
```

Output (first 5 of 5 row(s))

customer_id	customer_name	num_active_loans	total_loan_amount
26	Anthony Perez	2	752963.88
57	Shannon Pierce	2	719301.4099999999
105	Loretta Barnes	2	533704.88
127	Laura Grant	2	454377.42
183	Monica Smith	2	367885.54000000004

Insight: Only 5 customers currently hold multiple active loans, but their combined loan amounts are large (up to ~$753K). These multi-loan customers represent concentrated credit risk and should be reviewed individually for debt-to-income and repayment-capacity checks before any further credit is extended to them.

III. Branch Performance Analysis

III.A — Top 2 branches by new accounts opened, per year

SQL Query

```
WITH yearly AS (
  SELECT
    b.branch_name,
    YEAR(a.opened_date) AS yr,
    COUNT(a.account_id) AS num_accounts
  FROM citibank.accounts a
```

```
  JOIN citibank.branches b ON b.branch_id = a.branch_id
  WHERE a.opened_date IS NOT NULL
  GROUP BY b.branch_name, yr
),
ranked AS (
  SELECT *, RANK() OVER (PARTITION BY yr ORDER BY num_accounts DESC) AS rnk
  FROM yearly
)
SELECT branch_name, yr AS year, num_accounts
FROM ranked
WHERE rnk <= 2
ORDER BY yr, num_accounts DESC;
```

Output (first 10 of 21 row(s))

branch_name	year	num_accounts
Brickell Avenue	2018	5
West Palm	2018	5
Coral Gables	2019	3
Hialeah Main	2019	3
Downtown Miami	2020	5
Brickell Avenue	2020	4
Hialeah Main	2021	5
Downtown Miami	2021	4
West Palm	2021	4
Downtown Miami	2022	4

Insight: Leadership in new account acquisition rotates across different branches year to year (e.g., Brickell Avenue and West Palm led in 2018, Hialeah Main and Downtown Miami in later years), rather than being dominated by one location. This suggests account growth is being driven by local branch initiatives or staffing changes rather than a systemic advantage — successful branches should have their acquisition tactics documented and shared network-wide.

III.B — Branch performance ranked by transaction amount

SQL Query

```
WITH branch_stats AS (
  SELECT
    b.branch_id,
    b.branch_name,
    (SELECT COUNT(*) FROM citibank.employees e WHERE e.branch_id = b.branch_id) AS
num_employees,
    (SELECT COUNT(*) FROM citibank.accounts a WHERE a.branch_id = b.branch_id) AS num_accounts,
    COALESCE((SELECT SUM(t.amount) FROM citibank.transactions t
              JOIN citibank.accounts a ON a.account_id = t.account_id
              WHERE a.branch_id = b.branch_id), 0) AS total_transaction_amount
  FROM citibank.branches b
)
SELECT
  branch_name, num_employees, num_accounts, total_transaction_amount,
  RANK() OVER (ORDER BY total_transaction_amount DESC) AS performance_rank
FROM branch_stats
ORDER BY performance_rank;
```

Output (first 10 of 12 row(s))

branch_name	num_employees	num_accounts	total_transaction_amount	performance_rank
Downtown Miami	14	28	195700.84	1
Kendall Branch	20	21	147653.05	2
West Palm	19	27	133676.33	3
Dade Central	18	20	119072.25	4
Brickell Avenue	16	20	116130.26	5
South Beach	21	19	96288.95	6
Aventura Plaza	10	18	88967.27	7
Hialeah Main	15	19	68486.79	8
Coral Gables	15	17	61614.41	9
Fort Lauderdale	14	5	31726.059999999998	10

Insight: Downtown Miami leads all branches in transaction volume (~$195.7K) despite having fewer employees (14) than several lower-ranked branches — while Key Biscayne recorded $0 in transaction amount despite having 20 employees. Staffing levels are clearly not the primary driver of branch performance; Key Biscayne in particular warrants an operational review to understand why it is generating no transaction activity.

III.C — Branch yearly transaction totals, cumulative totals, and network average

SQL Query

```
WITH branch_year AS (
  SELECT
    b.branch_name,
    YEAR(t.transaction_date) AS yr,
    SUM(t.amount) AS total_transaction_amount
  FROM citibank.transactions t
  JOIN citibank.accounts a ON a.account_id = t.account_id
```

```
  JOIN citibank.branches b ON b.branch_id = a.branch_id
  GROUP BY b.branch_name, yr
)
SELECT
  branch_name,
  yr AS year,
  total_transaction_amount,
  SUM(total_transaction_amount) OVER (PARTITION BY branch_name ORDER BY yr) AS
cumulative_transaction_amount,
  ROUND(AVG(total_transaction_amount) OVER (PARTITION BY yr), 2) AS
avg_transaction_amount_per_year_all_branches
FROM branch_year
ORDER BY branch_name, yr;
```

Output (first 10 of 59 row(s))

branch_name	year	total_transaction_amount	cumulative_transaction_amount	avg_transaction_amount_per_year_all_branches
Aventura Plaza	2020	769.2	769.2	17645.23
Aventura Plaza	2021	33558.09	34327.28999999999	22621.19
Aventura Plaza	2022	4542.62	38869.909999999996	14001.76
Aventura Plaza	2023	12254.4	51124.31	19845.94
Aventura Plaza	2024	11487.89	62612.2	15753.11
Aventura Plaza	2025	26355.07	88967.26999999999	18531.45
Brickell Avenue	2020	35947.51	35947.51	17645.23
Brickell Avenue	2021	33016.94	68964.45000000001	22621.19
Brickell Avenue	2022	24709.579999999998	93674.03	14001.76
Brickell Avenue	2023	12382.83	106056.86	19845.94

Insight: Comparing each branch's cumulative transaction total against the network-wide yearly average highlights which branches are consistently under-performing their peers over time versus branches with a single strong year. This trended view is better suited to setting branch-level growth targets than a single-year snapshot, since it separates one-off spikes from sustained performance.

III.D — Branch loan portfolios (amount issued, active loans, rate, % paid off)

SQL Query

```
SELECT
  b.branch_id,
  b.branch_name,
  SUM(l.loan_amount) AS total_loan_amount_issued,
  SUM(CASE WHEN l.status = 'ACTIVE' THEN 1 ELSE 0 END) AS num_active_loans,
  ROUND(AVG(l.interest_rate), 2) AS avg_interest_rate,
  ROUND(100 * SUM(CASE WHEN l.status = 'PAID' THEN 1 ELSE 0 END) / COUNT(*), 2) AS
pct_loans_paid
FROM citibank.loans l
JOIN citibank.branches b ON b.branch_id = l.branch_id
GROUP BY b.branch_id, b.branch_name
ORDER BY total_loan_amount_issued DESC;
```

Output (first 10 of 12 row(s))

branch_id	branch_name	total_loan_amount_issued	num_active_loans	avg_interest_rate	pct_loans_paid
4	Kendall Branch	6933541.74	7	8.12	45.83
9	West Palm	6347648.03	11	9.04	25.0
5	Dade Central	5899474.11	8	9.81	23.81
1	Downtown Miami	5777293.87	8	9.27	29.17
7	Hialeah Main	5665030.22	7	8.77	35.0
2	South Beach	5445106.45	6	8.28	36.84
6	Brickell Avenue	5361644.51	2	9.08	40.0
8	Aventura Plaza	4553607.41	5	8.84	45.0
3	Coral Gables	4489395.51	9	8.09	26.32
12	North Miami	1774444.92	1	8.43	60.0

Insight: Fort Lauderdale has issued loans but shows a 0% paid-off rate, while North Miami's small loan book (~$1.77M) has the highest repayment completion (60%). Loan book size and repayment health are not correlated — branches should be evaluated on loan quality (repayment rate) independently from loan volume when setting branch-manager incentives.

IV. Financial Risk and Revenue Analysis

IV.A — Top 3 branches by loan default rate, per loan type

SQL Query

```
WITH loan_stats AS (
  SELECT
    l.loan_type,
    b.branch_name,
    COUNT(l.loan_id) AS total_loans_issued,
    SUM(CASE WHEN l.status = 'DEFAULTED' THEN 1 ELSE 0 END) AS num_defaults,
    ROUND(100 * SUM(CASE WHEN l.status = 'DEFAULTED' THEN 1 ELSE 0 END) / COUNT(*), 2) AS
default_rate_pct
  FROM citibank.loans l
  JOIN citibank.branches b ON b.branch_id = l.branch_id
  GROUP BY l.loan_type, b.branch_name
),
ranked AS (
  SELECT *, RANK() OVER (PARTITION BY loan_type ORDER BY default_rate_pct DESC) AS rnk
  FROM loan_stats
)
SELECT loan_type, branch_name, total_loans_issued, num_defaults, default_rate_pct
FROM ranked
WHERE rnk <= 3
ORDER BY loan_type, default_rate_pct DESC;
```

Output (first 10 of 15 row(s))

loan_type	branch_name	total_loans_issued	num_defaults	default_rate_pct
AUTO	Brickell Avenue	5	3	60.0
AUTO	Aventura Plaza	4	2	50.0
AUTO	Dade Central	2	1	50.0
AUTO	Downtown Miami	6	3	50.0
AUTO	North Miami	2	1	50.0
BUSINESS	Fort Lauderdale	2	2	100.0
BUSINESS	Hialeah Main	6	4	66.67
BUSINESS	West Palm	8	5	62.5
MORTGAGE	Key Biscayne	1	1	100.0
MORTGAGE	Coral Gables	3	2	66.67

Insight: Default rates are alarmingly high in places — Fort Lauderdale (BUSINESS), Key Biscayne (MORTGAGE), and South Beach (PERSONAL) all show 100% default rates within their respective loan types. While sample sizes per branch/type are small, these combinations should be the first targets for a tightened underwriting review, since a 100% default rate on even a handful of loans signals a systemic approval or servicing failure rather than isolated bad luck.

IV.B — Branch contribution to total interest revenue (active loans)

SQL Query

```
WITH branch_interest AS (
  SELECT
    b.branch_name,
    SUM(l.loan_amount * l.interest_rate / 100) AS interest_revenue
  FROM citibank.loans l
  JOIN citibank.branches b ON b.branch_id = l.branch_id
  WHERE l.status = 'ACTIVE'
  GROUP BY b.branch_name
)
SELECT
  branch_name,
  ROUND(interest_revenue, 2) AS total_interest_revenue,
  ROUND(100 * interest_revenue / SUM(interest_revenue) OVER (), 2) AS
pct_contribution_to_total_revenue
FROM branch_interest
ORDER BY total_interest_revenue DESC;
```

Output (first 10 of 10 row(s))

branch_name	total_interest_revenue	pct_contribution_to_total_revenue
Downtown Miami	225664.13	16.12
West Palm	213951.32	15.28
Dade Central	206676.99	14.76
Coral Gables	175736.69	12.55
Kendall Branch	148500.64	10.61
Hialeah Main	139581.82	9.97
South Beach	120944.24	8.64
Aventura Plaza	108872.26	7.78
Brickell Avenue	44748.11	3.2
North Miami	15269.19	1.09

Insight: The top 3 branches (Downtown Miami, West Palm, Dade Central) generate over 46% of total active-loan interest revenue combined, while North Miami contributes just 1.09%. Interest revenue is heavily concentrated in a small number of branches, so any disruption at Downtown Miami, West Palm, or Dade Central would have an outsized effect on overall interest income — these branches merit closer risk monitoring and business-continuity planning.

IV.C — Active loans overdue >180 days or with no payment (as of April 30, 2025)

SQL Query

```
WITH last_payment AS (
  SELECT loan_id, MAX(payment_date) AS last_payment_date
  FROM citibank.loan_payments
  GROUP BY loan_id
)
SELECT
  l.loan_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  l.loan_amount,
  DATEDIFF('2025-04-30', COALESCE(lp.last_payment_date, l.start_date)) AS
days_since_last_payment
FROM citibank.loans l
JOIN citibank.customers c ON c.customer_id = l.customer_id
LEFT JOIN last_payment lp ON lp.loan_id = l.loan_id
WHERE l.status = 'ACTIVE'
  AND (
    lp.last_payment_date IS NULL
    OR DATEDIFF('2025-04-30', lp.last_payment_date) > 180
  )
ORDER BY days_since_last_payment DESC;
```

Output (first 10 of 49 row(s))

loan_id	customer_name	loan_amount	days_since_last_payment
189	Margaret Terry	231378.31	2648.0
61	Timothy Harrington	470793.83	2539.0
86	Charles Harrington	324169.73	2507.0
71	Monica Smith	224639.1	2351.0
66	Brandy Walker	232972.01	1794.0
130	Loretta Barnes	70748.73	1701.0
160	Sean Myers	111038.7	1664.0
48	Scott Smith	446620.39	1465.0
117	Audrey Tucker	228961.07	1444.0
64	Luke Miller	217328.18	1443.0

Insight: 49 active loans have gone unpaid or are more than 180 days overdue as of the reference date, some for over 2,600 days (roughly 7 years) since the last payment. These long-overdue active loans are very likely to be defaults that have simply not been reclassified — the collections/servicing team should prioritize a status review of these accounts, since they overstate the bank's healthy active-loan book.

IV.D — Year-over-year change in Jan-Aug transaction totals

SQL Query

```
WITH yearly AS (
  SELECT
    YEAR(t.transaction_date) AS yr,
    SUM(t.amount) AS total_amount
  FROM citibank.transactions t
  WHERE MONTH(t.transaction_date) BETWEEN 1 AND 8
  GROUP BY yr
),
with_prev AS (
  SELECT
    yr, total_amount,
    LAG(total_amount) OVER (ORDER BY yr) AS prev_year_amount
  FROM yearly
)
SELECT
  yr AS year,
  total_amount AS total_transaction_amount,
  prev_year_amount AS previous_year_total_amount,
  ROUND(100 * (total_amount - prev_year_amount) / prev_year_amount, 2) AS pct_increase
FROM with_prev
```

```
WHERE prev_year_amount IS NOT NULL
ORDER BY yr;
```

Output (first 5 of 5 row(s))

year	total_transaction_amount	previous_year_total_amount	pct_increase
2021	166949.56	123536.86	35.14
2022	123463.67	166949.56	-26.05
2023	137243.18	123463.67	11.16
2024	96727.81	137243.18	-29.52
2025	136672.03	96727.81	41.3

Insight: Jan-Aug transaction totals swing sharply year to year (e.g., +35.1% in 2021, -26.1% in 2022, +41.3% in 2025), with no clear sustained growth trend. This volatility suggests transaction volume is being driven by external or one-off factors rather than steady organic growth, and the bank should investigate what drove the 2022 and 2024 declines before assuming 2025's rebound will continue.

V. Customer Service and Interaction Analysis

V.A — Branch complaint rate, interaction volume, and staffing

SQL Query

```
WITH branch_interactions AS (
  SELECT
    b.branch_id,
    b.branch_name,
    COUNT(ci.interaction_id) AS total_interactions,
    SUM(CASE WHEN ci.interaction_type = 'COMPLAINT' THEN 1 ELSE 0 END) AS num_complaints
  FROM citibank.branches b
  LEFT JOIN citibank.employees e ON e.branch_id = b.branch_id
  LEFT JOIN citibank.customer_interactions ci ON ci.employee_id = e.employee_id
  GROUP BY b.branch_id, b.branch_name
),
branch_employees AS (
  SELECT
    branch_id,
    COUNT(*) AS num_employees,
    GROUP_CONCAT(DISTINCT position) AS positions
  FROM citibank.employees
  GROUP BY branch_id
)
SELECT
  bi.branch_name,
  bi.total_interactions,
  bi.num_complaints,
  ROUND(100 * bi.num_complaints / NULLIF(bi.total_interactions,0), 2) AS complaint_rate_pct,
  be.num_employees,
  be.positions
```

```
FROM branch_interactions bi
JOIN branch_employees be ON be.branch_id = bi.branch_id
ORDER BY complaint_rate_pct DESC;
```

Output (first 10 of 12 row(s))

branch_name	total_interactions	num_complaints	complaint_rate_pct	num_employees	positions
Brickell Avenue	13	6	46.15	16	Teller,Mortgage Specialist,Comp...
Hialeah Main	11	4	36.36	15	Branch Manager,Compliance Offic...
South Beach	28	10	35.71	21	Operations Manager,Branch Manag...
Downtown Miami	20	7	35.0	14	Compliance Officer,IT Support S...
Coral Gables	10	3	30.0	15	Customer Service Representative...
Kendall Branch	26	7	26.92	20	Operations Manager,Compliance O...
Aventura Plaza	13	3	23.08	10	Compliance Officer,Financial Ad...
Fort Lauderdale	13	3	23.08	14	Branch Manager,Accountant,Mortg...
Key Biscayne	18	4	22.22	20	Branch Manager,IT Support Speci...
Dade Central	11	2	18.18	18	Customer Service Representative...

Insight: Brickell Avenue has the highest complaint rate (46.2%) despite a mid-sized staff, while several branches with more employees have far lower complaint rates. Complaint rate does not track with headcount, which points to service-quality or training gaps at the highest-complaint branches rather than a simple staffing shortage — Brickell Avenue, Hialeah Main, and South Beach should be prioritized for a service-quality audit.

V.B — Customers who complained in 3 or more distinct months

SQL Query

```
WITH complaints AS (
  SELECT
    customer_id,
    DATE_FORMAT(interaction_date, '%Y-%m') AS complaint_month
  FROM citibank.customer_interactions
  WHERE interaction_type = 'COMPLAINT'
),
agg AS (
  SELECT
    customer_id,
    COUNT(DISTINCT complaint_month) AS num_complaint_months,
    COUNT(*) AS total_complaints,
    GROUP_CONCAT(DISTINCT complaint_month) AS months
  FROM complaints
  GROUP BY customer_id
  HAVING COUNT(DISTINCT complaint_month) >= 3
)
SELECT
  a.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  a.num_complaint_months,
  a.total_complaints,
  a.months
FROM agg a
JOIN citibank.customers c ON c.customer_id = a.customer_id
ORDER BY a.num_complaint_months DESC, a.total_complaints DESC;
```

Output (first 1 of 1 row(s))

customer_id	customer_name	num_complaint_months	total_complaints	months
198	Nathan Fernandez	3	3	2021-05,2024-06,2020-10

Insight: Only one customer in the dataset (Nathan Fernandez) has complained across 3 or more distinct months. While this is a very small chronic-complainer segment, this customer should be flagged for a proactive retention outreach and root-cause review before churn occurs, since repeat complaints over multiple months are a strong churn predictor even at low volume.

VI. Card Portfolio Optimisation Analysis

VI.A — Active credit cards with balance ≥90% of credit limit

SQL Query

```
SELECT
  c.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  cd.card_number,
  cd.credit_limit,
  a.balance,
  ROUND(100 * a.balance / cd.credit_limit, 2) AS pct_credit_used
FROM citibank.cards cd
JOIN citibank.accounts a ON a.account_id = cd.account_id
JOIN citibank.customers c ON c.customer_id = a.customer_id
WHERE cd.card_status = 'ACTIVE'
  AND cd.card_type = 'CREDIT'
  AND cd.credit_limit IS NOT NULL
  AND a.balance / cd.credit_limit >= 0.90
ORDER BY pct_credit_used DESC;
```

Output (first 10 of 22 row(s))

customer_id	customer_name	card_number	credit_limit	balance	pct_credit_used
83	Gary Cunningham	3570866882634353.0	2303.75	98077.46	4257.3
62	Kenneth Morrison	4105123828258673.0	1327.62	43987.26	3313.24
146	Abigail Cole	570541152627.0	4867.61	43915.34	902.2
164	Amanda Ball	30464887719062.0	8810.94	69031.35	783.47
83	Gary Cunningham	30437489691087.0	13610.09	98077.46	720.62
106	Mary Scott	4215107029662.0	12470.56	86383.67	692.7
83	Gary Cunningham	4752500743292717.0	14161.39	98077.46	692.57
179	Amy Bates	4458685014294019.0	8588.61	55441.25	645.52
151	Joan Johnson	30223320616321.0	10167.49	59679.07	586.96
81	Shannon Jordan	3596705466889373.0	10645.59	60620.89	569.45

Insight: 22 active credit card accounts show a balance-to-credit-limit ratio at or above 90% — and in many cases the linked account balance far exceeds the card's credit limit (over 4,000% in the most extreme case). This large gap indicates the 'balance' field being compared is the customer's overall account balance rather than the card's outstanding statement balance; before this metric is used for real credit-limit decisions, the data model should be extended to track actual card outstanding balances separately from account balances.

VI.B — Monthly spend by card type in 2025 (active cards)

SQL Query

```
SELECT
  cd.card_type,
  MONTH(t.transaction_date) AS month_num,
  ROUND(SUM(t.amount), 2) AS total_amount_spent,
  COUNT(DISTINCT a.customer_id) AS unique_cardholders,
  ROUND(AVG(t.amount), 2) AS avg_transaction_size
FROM citibank.transactions t
JOIN citibank.accounts a ON a.account_id = t.account_id
JOIN citibank.cards cd ON cd.account_id = a.account_id
WHERE cd.card_status = 'ACTIVE'
  AND YEAR(t.transaction_date) = 2025
GROUP BY cd.card_type, month_num
ORDER BY cd.card_type, month_num;
```

Output (first 10 of 11 row(s))

card_type	month	total_amount_spent	unique_cardholders	avg_transaction_size
CREDIT	03	7509.32	1	3754.66
CREDIT	04	8973.11	1	8973.11
CREDIT	06	5430.69	1	5430.69
DEBIT	01	9662.33	1	9662.33
DEBIT	02	6909.19	1	6909.19
DEBIT	03	6951.1	2	3475.55
DEBIT	05	11555.5	1	5777.75
DEBIT	08	11302.71	2	5651.35
DEBIT	09	9514.91	2	4757.45
DEBIT	10	5171.11	1	5171.11

Insight: Debit card spend is both higher in total volume and more consistently active month-to-month than credit card spend in 2025, with several months showing no active credit card transactions at all. This suggests the credit card portfolio is underutilized relative to debit — a targeted rewards or cashback campaign could help shift more spend onto credit cards, which are typically more profitable for the bank through interchange and interest income.

VI.C — Transactions by card type and transaction type

SQL Query

```
SELECT
  cd.card_type,
  t.transaction_type,
  COUNT(t.transaction_id) AS num_transactions,
  ROUND(SUM(t.amount), 2) AS total_transaction_amount,
  ROUND(AVG(t.amount), 2) AS avg_transaction_amount
FROM citibank.transactions t
JOIN citibank.accounts a ON a.account_id = t.account_id
JOIN citibank.cards cd ON cd.account_id = a.account_id
GROUP BY cd.card_type, t.transaction_type
ORDER BY cd.card_type, num_transactions DESC;
```

Output (first 8 of 8 row(s))

card_type	transaction_type	num_transactions	total_transaction_amount	avg_transaction_amount
CREDIT	TRANSFER	45	255069.1	5668.2
CREDIT	WITHDRAWAL	29	200650.1	6918.97
CREDIT	DEPOSIT	25	114793.42	4591.74
CREDIT	PAYMENT	24	136852.09	5702.17
DEBIT	TRANSFER	43	214485.0	4988.02
DEBIT	PAYMENT	21	120273.69	5727.32
DEBIT	WITHDRAWAL	20	90388.15	4519.41
DEBIT	DEPOSIT	15	95202.27	6346.82

Insight: TRANSFER is the most common transaction type on both CREDIT and DEBIT cards, followed by WITHDRAWAL/PAYMENT activity. Credit cards show a higher average transaction size on WITHDRAWAL-type activity than debit cards, which combined with the low credit utilization seen in VI.B suggests credit card usage today is skewed toward occasional large transactions rather than everyday spending — another sign the card products are not yet embedded in customers' daily spending habits.

VII. Actionable Insights & Recommendations

The sections below synthesize the key findings from each analysis area into practical, data-driven recommendations, organized around three themes: addressing low engagement with targeted incentives, mitigating high-risk exposure, and replicating the practices of top-performing branches and segments.

A. Customer Behavior — Deepening Engagement

- Activate high-balance, low-activity accounts: High Balance customers (avg. ~$77.6K) transact the least of any balance tier (II.A). Cross-sell cash-management tools, sweep accounts, and premium rewards cards to convert idle balances into active relationships.
- Protect the top-value segment: The top 10 customers by balance (II.D) and the 5 customers with multiple active loans (II.E) represent concentrated value and concentrated risk. Assign dedicated relationship managers to the former and run individual credit reviews on the latter.
- Build credit habits earlier: Customers under 30 generate the fewest credit card transactions of any age group (II.C). A starter credit card with engagement-based rewards can build usage habits and lifetime value earlier in the relationship.
- Align operations to seasonality: Each account type peaks in a different month in 2025 (II.B) — align staffing, fraud monitoring, and marketing pushes to account-type-specific peaks rather than a single bank-wide calendar.

B. Branch Performance — Replicating What Works

- Investigate underperforming branches: Key Biscayne recorded $0 in transaction volume despite having 20 employees (III.B), and Fort Lauderdale shows a 0% loan repayment rate (III.D). Both warrant an operational review before further investment.
- Codify and share best practices: Leadership in new-account growth rotates across branches year to year (III.A) rather than concentrating in one location — document what top branches do differently each year and roll it out network-wide.
- Use trend, not snapshot, performance: The cumulative and network-average views (III.C) separate branches with sustained growth from those with a single strong year; use this trended view, not a single-year total, when setting branch targets.

C. Financial Risk & Revenue — Managing Exposure

- Tighten underwriting where default rates are highest: Fort Lauderdale (BUSINESS), Key Biscayne (MORTGAGE), and South Beach (PERSONAL) each show 100% default rates within their loan type (IV.A) — pause new originations of that type at that branch pending review.
- Reclassify stale "active" loans: 49 active loans are overdue more than 180 days, some for ~7 years (IV.C). These should be reviewed and likely reclassified as defaults so the active-loan book is not overstated.
- Plan around revenue concentration: The top 3 branches generate over 46% of active-loan interest revenue combined (IV.B) — build business-continuity plans specifically around Downtown Miami, West Palm, and Dade Central.
- Investigate transaction volatility: Jan–Aug transaction totals swing sharply year over year with no sustained trend (IV.D) — determine what drove the 2022 and 2024 declines before assuming the 2025 rebound continues.

D. Customer Service — Reducing Complaints and Churn Risk

- Audit service quality at high-complaint branches: Brickell Avenue, Hialeah Main, and South Beach have the highest complaint rates (V.A), independent of staffing levels — the root cause is likely training or process quality, not headcount.
- Proactively manage repeat complainers: Customers who complain across 3+ distinct months (V.B) are a strong churn signal even in small numbers — route them to a retention specialist for root-cause resolution before they leave.

E. Card Portfolio — Growing Profitable Usage

- Shift spend toward credit: Debit card spend is higher and more consistently active month-to-month than credit card spend in 2025 (VI.B), with some months showing no active credit transactions at all. A rewards/cashback campaign can shift everyday spend onto the more profitable credit product.
- Fix balance-vs-limit tracking before acting on utilization: The >=90%-utilization list (VI.A) shows ratios well over 100% because account balance is compared against card credit limit rather than actual card outstanding balance. Extend the data model to track card statement balances separately before using this metric for real credit-limit decisions.
- Encourage everyday card usage: TRANSFER and WITHDRAWAL dominate card transaction types (VI.C) over routine PAYMENT activity, suggesting cards are not yet embedded in daily spending — everyday-spend incentives (grocery/fuel cashback) could shift this mix.